Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Chair of Computer Science VIII
Aerospace Information Technology
Prof. Dr.-Ing. Sergio Montenegro

# Introduction to RODOS

## Online-Floatsat

***Frank Flederer***

***E-Mail: frank.flederer@uni-wuerzburg.de***

***Tel.: +49 931 31-85120***

# Why should we use a RTOS?

- Systems have several tasks with
  - different timing constraints and
  - different priorities.

  ➔ Dynamic scheduling
- We want to develop for a target platform and a testing/development platform.

  ➔ Hardware abstraction

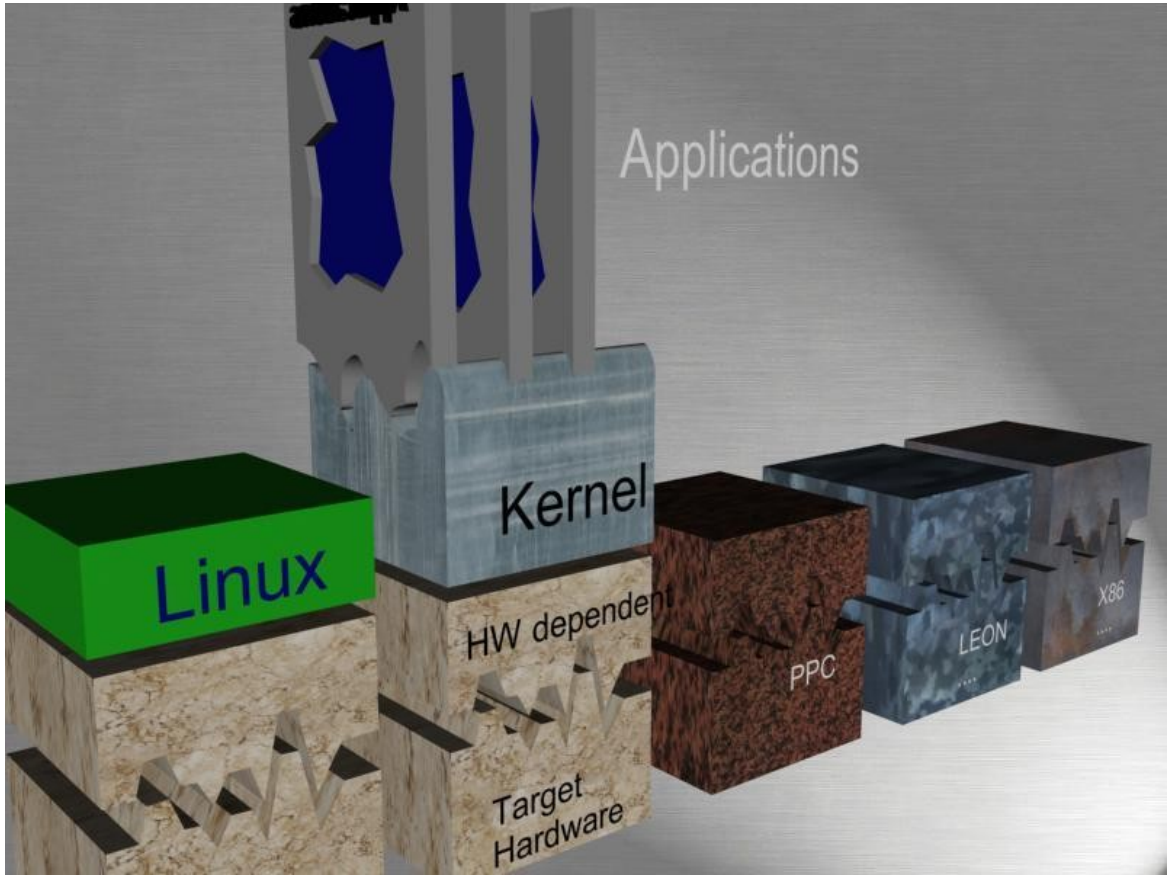
# Why should we use a RTOS?

- Clear programming interface.
- Clear communication mechanisms.

  ➔ Divide programming effort to parts that can be developed independently.

# Rodos

- **R**eal-time **O**n-board **D**ependable **O**perating **S**ystem
- Preemptive dynamic scheduled
- Written (mostly) in C++ (and some assembler)
- Supports several platforms
  - Bare-metal: Raspberry Pi, STM32F4 (e.g. Discovery), Smartfusion 2, Gecko
  - On other OS: Posix


Applications
Kernel
Linux
HW dependent
Target
Hardware
PPC
LEON
X86

# Directory structure

```
rodos
├── api
│   ├── gateway
│   └── hal
├── cmake
│   └── port
├── doc
├── scripts
│   ├── build-scripts
│   └── help-scrips
├── src
│   ├── bare-metal
│   │   ├── linux-makecontext
│   │   ├── linux-x86
│   │   ├── raspberrypi3
│   │   ├── sf2
│   │   ├── stm32f4
│   │   └── template
│   ├── bare-metal-generic
│   │   └── ip
│   ├── independent
│   │   └── gateway
│   ├── on-posix
│   └── on-posix-mac
```

- Declaration headers (*.h)
- Cmake configuration files
- Documentation
- Build scripts
- Bare-metal source files - dependent of the platform
- Bare-metal source files - idependent of the platform
- High-level source files - idependent of the platform

# Directory structure

```
├── support
│   ├── support-libs
│   │   └── ccsds
│   └── support-programs
│       ├── maskgenerator
│       ├── middleware-standalone
│       ├── serializer
│       ├── telecommand-generator
│       ├── timing-display-octave
│       └── udpinterface
├── test-suite
└── tutorials
    ├── 10-first-steps
    ├── 20-core
    ├── 30-communication-and-bbs
    ├── 40-support-lib
    ├── 50-time-related
    ├── 60-tutorial-big-application
    └── 70-network-and-io
```

Optional useful libraries

Optional useful programs

Automatic tests

Tutorials

# Compiling

- CMake
  - Useful when using CMake-based tools, e.g. CLion
- Build scripts
  - Useful to quickly compile programs, without the need to write configuration files

# Compiling

- Load environment

```
cd path/to/rodos
source setenvs.sh
```

- Compile Rodos lib

```
rodos-lib.sh linux-x86
```

  - Parameter is the platform to compile for

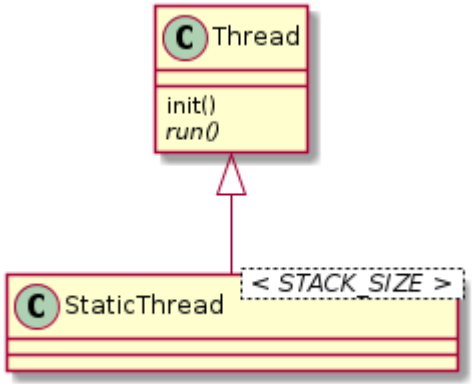
# Compiling

- Compile your software

```
rodos-executable.sh linux-x86 my-first-file.cpp my-second-file.cpp
```

  - The first parameter is the platform to compile for.
  - The other parameters are the source files that should be included into the executable.

# Thread




```
class MyThread : public StaticThread<> {
private:
  int myMemberVariable = 1;

public:
  void init() override {
    myMemberVariable = readMyValue();
  }

  void run() override {
    TIME_LOOP(1200 * MILLISECONDS, 1 * SECONDS) {
      myMemberVariable += 10;
      RODOS::PRINTF("My Member Variable Value: %d\n", myMemberVariable);
    }
  }
};

static MyThread myThread;
```

Create your thread subclass implementing your intended behavior

Init stuff here before starting the thread.

The thread‘s code goes here.

Timed loop (first run, period)

(Debug) Printing, just like fprint().

Initialize the thread. That‘s it, no manual registering necessary.

# Timing

- Internal representation: nanoseconds
- Time literals

```
constexpr int64_t NANOSECONDS  = 1LL;
constexpr int64_t MICROSECONDS = (1000LL * NANOSECONDS);
constexpr int64_t MILLISECONDS = (1000LL * MICROSECONDS);
constexpr int64_t SECONDS      = (1000LL * MILLISECONDS);
constexpr int64_t MINUTES      = (60LL * SECONDS);
constexpr int64_t HOURS        = (60LL * MINUTES);
constexpr int64_t DAYS         = (24LL * HOURS);
constexpr int64_t WEEKS        = (7LL * DAYS);
```

- Retrieve current time: `NOW()`

# Timing

- Suspend the currently running thread for a given time:

```
RODOS::AT(NOW() + 2 * SECONDS)
```

  - The parameter is an absolute point in time, no time range!

- Suspend the thread forever… or until it is resumed manually:

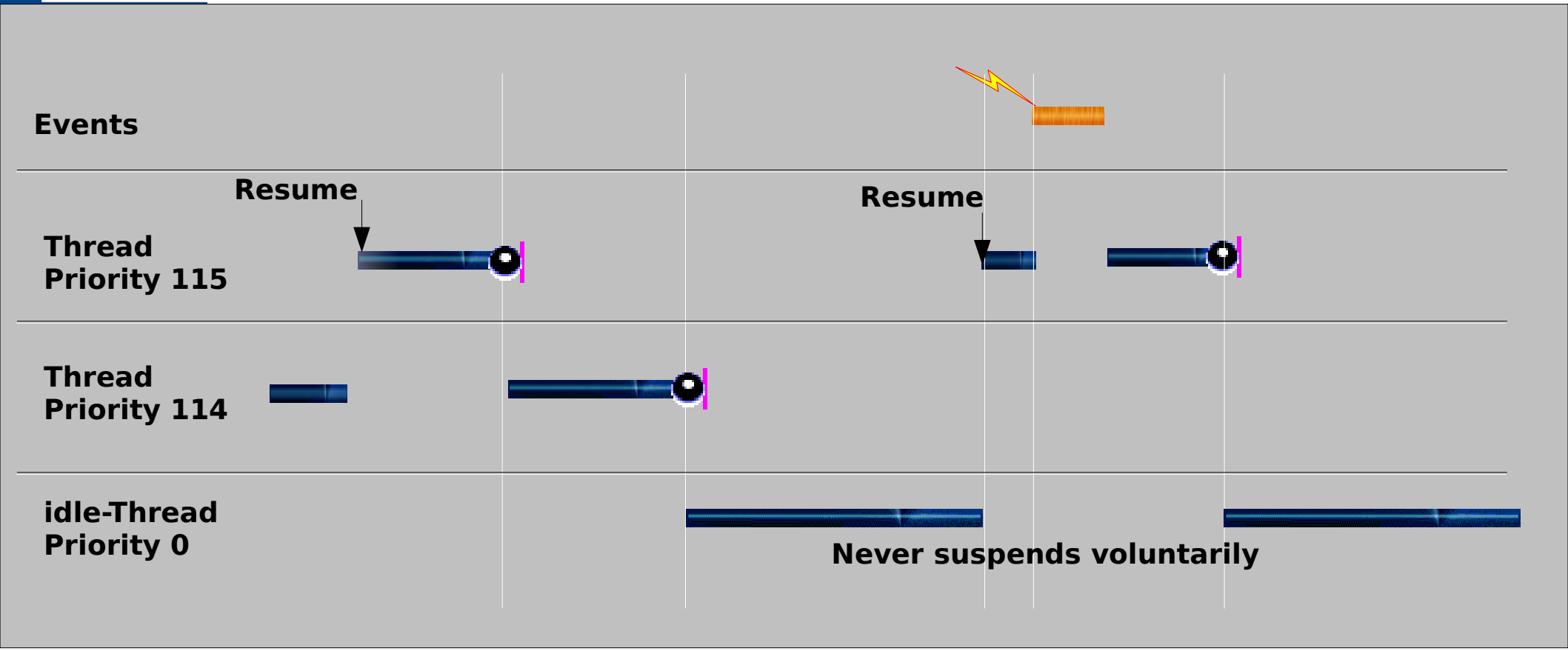
```
RODOS::AT(RODOS::END_OF_TIME)
```

# Timing

- Resume a suspended thread manually

```
myThread.resume();
```

# Scheduling


Events
Resume
Resume
Thread
Priority 115
Thread
Priority 114
idle-Thread
Priority 0
Never suspends voluntarily

# Useful functions

```
uint16_t checkSum(const void *buf, size_t len);

uint8_t someData[100];
auto myCheckSum = checkSum(someData, 100);
```

```
inline bool getbit(uint32_t bitmap, uint8_t bitIndex);

getbit(0xFEFE, 1); // returns true
getbit(0xFEFE, 0); // returns false
```

```
inline void setbit(uint32_t* bitmap, bool value, uint8_t bitIndex);

uint32_t bitmap = 0xFEFE;
setbit(&bitmap, 1, 0); // after that, bitmap is 0xFFFE
setbit(&bitmap, 0, 4); // after that, bitmap is 0xEFFE
```

# Serializing data

```
char buffer[100];
uint32_t numberOfBytes = RODOS::BasicSerializers::serialize(myData, buffer);
```

```
char buffer[100];
uint32_t i = 0
i = i + BasicSerializers::serialize(myFirstData, &buffer[i]);
i = i + BasicSerializers::serialize(mySecondData, &buffer[i]);
i = i + BasicSerializers::serialize(myThirdData, &buffer[i]);
```

# Deserializing data

```
char buffer[100];
receiveYourDataFromSomewhere(buffer);
uint32_t numberOfBytes = RODOS::BasicSerializers::deserialize(myData, buffer);
```

```
char buffer[100];
uint32_t i = 0
i = i + BasicSerializers::deserialize(myFirstData, &buffer[i]);
i = i + BasicSerializers::deserialize(mySecondData, &buffer[i]);
i = i + BasicSerializers::deserialize(myThirdData, &buffer[i]);
```

- Data variables are passed by reference; the value is overwritten.

# Exercise

- Run four threads that print their name and the current time in a given period.

  There should be a thread for each of those periods: 1, 2, 5, and 11.

# Thank you for your attention!

Q&A sessions on

- Monday 14:00 UTC

***Frank Flederer***

***E-Mail: frank.flederer@uni-wuerzburg.de***

***Tel.: +49 931 31-85120***